BULKING APP • AI MIGRATION PACKAGE

Project Identity & AI Development Instructions

Document 01 of 07 • Version 1.1

Purpose of this document
Give a new AI model a clear, durable understanding of what this project is, why it exists, what must remain true, how the owner expects development to be carried out, and how to make safe progress without unnecessary supervision.

Project	Personal bulking, nutrition, fitness and habit-management application
	One primary user; this is a personal tool, not a commercial multi-user product
	Python, Streamlit, Neon PostgreSQL, Gemini/OpenAI adapters (current implementation must be verified)
	Personalised, practical, data-grounded assistance with reliable manual workflows
	21 October 2026

Important evidence rule: This document records product intent and established decisions from project history. It is not a live repository audit. Current file paths, live database schema, working features, test results, deployment state and phase completion must be verified from the actual systems before code changes.

This document intentionally covers project identity and AI operating rules. Detailed personal profile values, the full database dictionary, feature acceptance criteria, phase status and deployment runbooks belong in the other documents in the seven-document handover package.

1. Project identity

The Bulking App is a private, personal application intended to help its owner consistently gain body mass through practical food planning, meal tracking, fitness routines and evidence-based adjustment. It should reduce daily decision-making and tracking effort rather than create a burdensome logging system.

The product is not simply a calorie calculator, a static meal plan, a generic chatbot or a public fitness platform. Its value comes from connecting the user's real preferences and circumstances with the food available, the meals planned, what was actually consumed, daily check-ins and longer-term progress.

1.1 Mission statement

Help the user follow a manageable, affordable and personalised routine for gradual weight gain by combining structured records, useful planning, simple workflows and careful analysis of real progress.

1.2 Product outcomes

- Reduce the effort required to decide what to eat and prepare.
- Generate useful plans based on the user's current profile, available food, time, preferences and practical constraints.
- Make it easy to record actual consumption without confusing it with planned meals.
- Maintain reliable history so weight, intake, digestion, exercise and adherence can be analysed over time.
- Use AI where it adds value, while keeping ordinary operations dependable without AI.
- Allow the application to evolve incrementally without repeatedly rebuilding existing functionality.
- Enable another AI or developer to understand the project and continue work from verified evidence.

1.3 Primary user

The primary user is the project owner. Requirements should be optimised for one person's real routine and preferences, not for hypothetical users, enterprise workflows or commercial scale. The app should nevertheless keep preferences configurable where practical, so changes in routine or goals do not require code edits.

1.4 Definition of a successful product

- The user can quickly see a relevant next action for the day.
- A saved plan and recorded history persist correctly across app restarts.
- Food suggestions respect explicit dietary restrictions and use known inventory and recipes when available.
- Actual meal events are retained even if plans are regenerated or edited.
- The user can correct records and understand the effect of corrections.
- The app explains uncertainty and data gaps rather than inventing facts.
- Features are documented and tested well enough that future development can proceed safely.

2. Product principles and non-negotiables

The following principles govern product and engineering decisions. A proposed implementation should be judged against these principles before adding complexity or changing established behaviour.

2.1 Personalisation must be based on evidence

- Use explicitly recorded profile preferences and user-provided facts.
- Use current data where available and show when information is missing or stale.
- Do not treat an old measurement, meal pattern or inferred preference as permanently current.
- Do not fabricate consumed food, ingredient availability, nutritional values, completed reminders or successful database writes.
- Make important recommendations explainable in terms of the inputs and constraints that influenced them.

2.2 Reliability before cleverness

- Correct persistence and data integrity are more important than sophisticated AI behaviour.
- Simple deterministic code is preferred for known operations, calculations, validation and database updates.
- AI-generated plans must be validated before they are saved or used to change records.
- An AI provider outage must not disable unrelated manual tracking or access to saved information.
- A feature is not complete merely because its UI exists; its behaviour, persistence and error handling must be checked.

2.3 Incremental development

- Prefer small, reviewable changes with clear acceptance criteria.
- Preserve working features unless a replacement is justified and tested.
- Avoid large rewrites, speculative abstractions and unrequested technology migrations.
- Keep documentation aligned with implementation and update it when decisions change.
- Do not begin a dependent task until its prerequisites are understood.

2.4 User control

- The user must be able to review and edit important plans and personal settings.
- Generated recommendations should not silently overwrite a confirmed plan.
- Destructive or difficult-to-reverse actions require explicit approval or a clearly documented, pre-approved policy.
- The system should make it possible to correct mistakes rather than trapping incorrect data in history.

3. Scope and boundaries

3.1 Intended scope

- Personal profile and preferences.
- Daily check-ins, including weight and selected routine or digestion variables.
- Food inventory, quantities, perishable items and CSV import/export.
- Recipe library with ingredients, preparation complexity and nutrition information.
- Meal plans that can be reviewed, edited, saved and regenerated.
- Meal logs representing what was actually consumed.
- Fitness and calisthenics planning, habit tracking and reminders.
- Historical charts and analysis based on records the user has supplied.
- Voice and conversational interaction where supported by the current implementation.
- AI provider adapters for optional planning or interpretation.
- Automated tests, useful documentation and a maintainable development process.

3.2 Out of scope unless the owner explicitly approves a change

- Commercial multi-user support, user accounts, authentication, billing or subscription management.
- Unrequested replacement of Streamlit or migration to a new database or hosting platform.
- Food-photo recognition or other major features not already established in requirements.
- Automatic purchasing, payments or other external irreversible actions.
- Medical diagnosis, treatment decisions or unsupported claims about supplements.
- A complex multi-agent framework simply because it is available or popular.
- Unreviewed destructive database operations, force-pushes or production changes.

3.3 How to handle proposed enhancements

When a new feature is suggested, classify it as (a) a required feature, (b) a useful but optional enhancement, or (c) an implementation detail. Explain the user value, effort, maintenance cost, new dependencies and effect on existing behaviour. Do not let optional enhancements delay the next required feature without a clear reason.

4. Personalisation: principles the app must preserve

The separate Personal Profile & Personalisation document will hold detailed and editable profile information. This section records the high-level rules that shape the product.

4.1 Established preferences and context

Dimension	Established context	Implication for the product
Diet	Vegetarian; eggs allowed	Exclude meat and fish; keep dietary rules editable.
Cuisine	Indian foods and familiar ingredients preferred	Prioritise practical, locally familiar meals.
Preparation	Simple meals preferred	Include effort and complexity in recipe selection.
Goal	Wants to gain body mass	Support gradual, monitored progress; do not hardcode unsafe targets.
Digestive observations	Has reported bloating with some shake ingredients	Record observations cautiously; do not diagnose or claim causation.
Exercise	Calisthenics; pull-up bar and dip bar	Reflect available equipment and recovery.
Budget	Value-conscious food and supplement choices	Consider cost while maintaining nutritional and safety standards.

4.2 Health-related caution

Earlier project context included a weight of about 40 kg, height of 172 cm and age of 21. These details may be outdated and must be confirmed in the app or in the separate profile document before being used as current values. If accurate, the combination warrants advice from a qualified clinician or dietitian. The app should not prescribe aggressive weight-gain targets, diagnose digestive issues or encourage supplement escalation without appropriate caution.

4.3 The product should adapt, not stereotype

- A previously used meal schedule is a starting point, not a permanent rule.
- Preferences and tolerances should be editable and revisable.
- A single missed meal should not trigger an extreme plan adjustment.
- A single weight measurement should not be treated as a reliable trend.
- The app should distinguish user observations from medical conclusions and AI guesses.

5. Established technical direction

The following choices have been established in project discussions. They are not a substitute for inspecting the repository. If the current code differs, the incoming AI must record the discrepancy and determine whether it is an implementation defect or an approved change.

Area	Established direction	Instruction
Application	Personal project; no authentication requirement	Do not add multi-user complexity without approval.
UI	Streamlit	Preserve current UI stack unless there is evidence it cannot meet a requirement.
Language	Python	Use the current environment and dependency conventions.
Database	Neon PostgreSQL selected; SQLite used earlier	Inspect live schema and code before changing persistence.
AI providers	Gemini and OpenAI adapters	Keep provider-specific code isolated and validate responses.
Inventory	CSV upload and downloadable template requested	Verify current behaviour and preserve working flows.
Meal records	Actual consumption separate from meal plans	Never conflate planned and consumed states.

5.1 Core domain concepts

- **Personal profile:** user-specific settings and relatively stable preferences.
- **Daily check-in:** a dated snapshot of measurements and selected daily variables.
- **Food inventory:** what is currently available, with quantities and units.
- **Recipe library:** reusable definitions of ingredients, preparation and estimated nutrients.
- **Meal plan:** an intended schedule of meals, not proof that they were eaten.
- **Meal log:** an event recording food actually consumed, with quantity and time where available.
- **Progress history:** recorded values and derived trends, with provenance preserved.

5.2 Data integrity rules

- A plan must not be counted as consumed without a real logging action or explicit user confirmation.
- Regenerating a plan must not delete historical meal-log events.
- Inventory changes must follow explicit rules and avoid double deductions from repeated submissions.
- Quantities must include units; incompatible units must not be added together.
- Corrections to logs must have a defined effect on totals and inventory.
- Database writes should be transactional where multiple related records must remain consistent.
- Numeric precision and valid ranges must be appropriate for weight and food quantities.
- Migrations must protect existing records and include a recovery plan for risky changes.

5.3 AI versus deterministic logic

Use deterministic code for input validation, known commands, arithmetic, database persistence, inventory arithmetic, timestamps, permission checks and repeatable transformations. Use AI for tasks such as interpreting natural-language requests, generating candidate meal plans, suggesting substitutions or explaining trends. Validate any AI output before it can affect persistent state.

6. Instructions for the incoming AI

Treat this document as the baseline for project identity and working principles. The repository, live database and more specific approved specifications provide the evidence for implementation details.

6.1 Before any implementation

1. Read this document and any available handover, roadmap, architecture and requirements documents.
2. Inspect the actual Git repository, active branch, current commit, working-tree status, directory structure, dependencies and tests.
3. Inspect the relevant database schema using authorised, read-only access first. Do not print secrets.
4. Identify the earliest incomplete task whose dependencies are satisfied.
5. State the proposed scope, affected files/data, risks and acceptance criteria before making a substantial change.
6. Search existing documentation, tests and history before asking the owner to repeat a decision.

6.2 During implementation

1. Make the smallest coherent change that satisfies the requirement.
2. Preserve existing architecture and conventions unless there is a concrete reason to change them.
3. Validate all inputs, imported CSV content, units, dates and AI-generated structured output.
4. Use parameterised SQL and appropriate transactions.
5. Keep credentials and environment-specific secrets out of code, logs, screenshots and documentation.
6. Add or update automated tests for changed behaviour and important edge cases.
7. Avoid new frameworks and dependencies unless they solve a specific documented problem.
8. Do not silently change product scope, schema semantics or established user preferences.

6.3 Before declaring work complete

1. Run the relevant tests and record the actual commands and results.
2. Verify persistence and integration behaviour, not just the visible UI.
3. Review the code diff for unrelated changes, secrets, accidental deletions and unsafe migrations.
4. Update the implementation status and affected documentation.
5. Commit only the intended changes when authorised and when required checks pass.
6. Report what changed, what was verified, what remains and the next actionable step.

7. Truthful status reporting

The AI must keep these states separate:

State	Meaning	Evidence required
Implemented	Code has been written	Relevant code and configuration exist.
Tested	Checks were executed	Actual test output or documented manual verification.
Committed	Changes are saved in a Git commit	Commit hash and status.
Pushed	Commit exists on the remote	Verified remote branch/commit.
Deployed	A deployment action completed	Deployment result and target environment.
Verified live	The deployed behaviour was checked	Successful post-deployment check.

Never claim a command, commit, push, migration or deployment succeeded because it was attempted. If a tool, permission or credential is missing, state the limitation plainly and give the next practical step.

8. Git, releases and destructive actions

- Inspect Git status before editing and before committing.
- Do not discard the owner's uncommitted changes.
- Use focused commits with descriptive messages.
- Use branches or worktrees to isolate risky or parallel agent work where practical.
- Do not force-push, rewrite shared history, delete branches or modify production data without explicit authorisation.
- For schema changes, inspect dependencies, prepare a migration, protect data and define rollback or recovery steps.
- Do not automatically deploy every successful local change unless that policy has been explicitly approved.

9. Autonomous development policy

The owner wants development to proceed with minimal manual intervention. This means the AI should autonomously perform routine, reversible engineering work when requirements and permissions are clear. It does not mean the AI has unrestricted authority over data, costs, credentials or production.

9.1 Recommended development loop

1. Read the task queue and current implementation status.
2. Select the next ready task and identify its acceptance criteria.
3. Inspect affected code, schema and existing tests.
4. Plan a small implementation and identify risk level.
5. Implement in an isolated branch/worktree where useful.
6. Run targeted tests and relevant regression tests.
7. Review the change for correctness, security, scope and maintainability.
8. Commit only after required checks pass and permissions allow it.
9. Update docs and status with evidence.
10. Continue to the next ready task, or stop when a genuine decision or blocker requires the owner.

9.2 Conditions that require stopping or asking

- An action may irreversibly delete or corrupt user data.

- A production migration, public deployment change or costly action lacks approval.
- Credentials or permissions required for the task are missing.
- Requirements conflict and cannot be reconciled through existing decisions.
- Tests reveal data corruption, a security problem or a serious regression.
- The authoritative environment or schema cannot be identified.
- A meaningful product trade-off depends on the owner’s preference.

9.3 Multi-agent coordination

- Assign each agent a distinct responsibility and bounded task.
- Avoid concurrent edits to the same files or database schema without coordination.
- Require a review step before merging agent-produced changes.
- Share only the context necessary for each task; keep project requirements consistent through versioned docs.
- Set retry and token/cost limits to prevent agents from looping indefinitely.
- Do not adopt CrewAI, LangChain, AG2, BeeAI or another framework by default. Choose tools only after defining the required workflow and evaluating cost, complexity and maintenance.

10. Communication standard

The owner prefers direct, concise, technically honest communication and step-by-step execution for development tasks. Avoid vague updates, repeated questions and lengthy descriptions of internal agent activity.

At the end of a meaningful development task, report:

- **Outcome:** what now works or what changed.
- **Files:** the important files changed and why.
- **Database:** migrations or schema changes, if any.
- **Verification:** commands run and actual outcomes.
- **Git:** branch, commit hash and whether changes were pushed.
- **Deployment:** whether deployment occurred and whether it was verified.
- **Remaining work:** defects, risks or blockers.
- **Next step:** the next logical task.

Do not ask the owner to run commands that the available authorised tools can safely execute. If local access or permissions are unavailable, explain exactly what is needed rather than implying the operation was performed.

11. Source-of-truth hierarchy

Use the following hierarchy when sources disagree, while respecting newer explicit owner decisions:

1. The owner’s current explicit instruction for the task.
2. Verified live behaviour and schema, for claims about what exists today.
3. Approved requirements and decision records, for intended behaviour.
4. Current source code and automated tests, interpreted together.
5. Git history and development-session records.
6. Older conversation summaries, assumptions or informal notes.

A mismatch between code and approved requirements must be documented. Do not silently redefine the requirement just because the current code behaves differently. Determine whether the mismatch is a defect, an approved change or an undocumented decision.

12. What this document does and does not establish

Topic	Established project context	Must be verified elsewhere
Repository	A GitHub repository named Bulk_App_Personalised was used	Exact remote URL, branch, HEAD, working tree and current permissions.
Stack	Streamlit and Python	Actual entry point, module structure, versions and launch command.
Database	Neon PostgreSQL was selected after SQLite was used initially	Current live schema, migration history and connection configuration.
AI providers	Gemini/OpenAI adapters were part of the plan/development	Current implementation, configured models, tests and failure handling.
CSV	Inventory CSV upload/template was requested and developed in prior work	Current behaviour, validation, duplicate handling and tests.
Voice	Speech input worked in Chrome during troubleshooting	Current browser support and actual field-mapping status.
Roadmap	Development was divided into phases	Exact completed, partial, broken and pending phase status.

13. How to use the seven-document handover

This is Document 1 of 7. It defines the product identity and operating principles; it does not replace the more detailed documents.

Document	Purpose
02. Personal Profile & Personalisation	Detailed profile values, food preferences, routine, equipment, budget and adaptation rules.
03. Technical Architecture & Database	Verified codebase map, system boundaries, live schema, data dictionary and integrations.
04. Functional Requirements & User Workflows	Feature behaviours, UI workflows, edge cases and acceptance criteria.
05. Development Roadmap & Current Status	Phase-by-phase evidence, blockers, dependencies and next tasks.
06. AI Development & Automation Strategy	Agent roles, workflow, autonomy, Git gates, model routing and cost controls.
07. Testing, Deployment & Troubleshooting	Test strategy, environment setup, secrets, release checks, known issues and recovery.

14. Copy-ready instruction for the new AI

Use this prompt when starting work with a new model. Provide this PDF and the other handover documents as they become available.

> You are taking over development of my personal Bulking App. Read this Project Identity and AI Development Instructions document before proposing changes. Follow the product principles, personalisation constraints, data integrity rules and operating safeguards described here.

Do not rewrite the app or replace its stack by default. First inspect the actual repository, current Git state, project files, tests, dependencies and the authorised live Neon PostgreSQL schema. Never print or commit secrets. Build a verified inventory of existing features and classify each as implemented, tested, committed, pushed, deployed, broken or incomplete, using evidence for each claim.

Preserve the personal-use scope, vegetarian-with-eggs preference, Indian-food focus, simple preparation, editable profile, calisthenics equipment constraints, separation of planned meals from actual meal logs, inventory integrity, CSV workflows and Gemini/OpenAI provider separation. Treat historical personal measurements and old implementation claims as requiring confirmation when relevant.

Then identify the earliest incomplete task whose dependencies are satisfied. State the scope and acceptance criteria, implement the smallest coherent change, test it, inspect the diff, protect existing data, update the documentation and report exactly what was verified. Never claim that something was tested, committed, pushed or deployed unless evidence confirms it. Ask me only when an important decision cannot be resolved from the repository or documentation, or when an action is destructive, costly, security-sensitive or otherwise requires my approval.

15. Maintenance rules

- Update this document only when project identity, scope boundaries or development rules materially change.
- Record why a material decision changed and link it to the decision log.
- Keep volatile implementation details in the status and architecture documents rather than treating them as permanent facts here.
- Do not place API keys, database passwords, access tokens or unnecessary sensitive personal data in this document.
- If a preference changes, update the authoritative personalisation document and any affected requirements.

End of Document 01 • Next: Personal Profile & Personalisation